S Selio · document de mise en service

Clés et configuration

Version du 16 octobre 2026 · branche `claude/gracious-wright-rf41nt` du dépôt selio

Démo Netlify
Aucune clé. Glisser le dossier de l'archive sur app.netlify.com/drop.

Mode connecté
Serveur Linux (API + worker), PostgreSQL 16, Redis : sections 2 et 3.

Votre modèle IA
Ollama + modèle de texte + modèle de vision : section 4.

Facultatif
Stripe pour facturer des abonnements : section 5.

1. Site web sur Netlify (vitrine + application)

À saisir dans Netlify › Site configuration › Environment variables. Ces variables sont PUBLIQUES (embarquées dans le site) : n'y mettez jamais de secret. Pour la démonstration seule, ne mettez rien.

VARIABLE	STATUT	EXEMPLE / FORMAT	À QUOI ELLE SERT
`VITE_API_BASE_URL`	Facultatif (démo) / Obligatoire (mode connecté)	`URL publique de votre API Selio, ex. https://api.votre-domaine.fr`	Vide = l'application ne propose que la démonstration (aucun serveur nécessaire).
`VITE_SHOW_INDICATIVE_PRICING`	Facultatif	`true ou false`	Affiche les tarifs indicatifs sur la vitrine.

2. Base de données et cache (serveur métier)

À placer dans le fichier .env du serveur (jamais dans Git ni dans Netlify). Avec deploy/docker-compose.prod.yml, PostgreSQL et Redis sont lancés pour vous : il suffit de choisir les deux mots de passe.

VARIABLE	STATUT	EXEMPLE / FORMAT	À QUOI ELLE SERT
`POSTGRES_PASSWORD`	Obligatoire	`mot de passe long (≥ 24 caractères)`	Mot de passe du propriétaire de la base « selio » (utilisé seulement pour les migrations).
`APP_DB_PASSWORD`	Obligatoire	`mot de passe long, différent du précédent`	Rôle applicatif « selio_app » (non superutilisateur) : l'isolation par organisation (RLS) s'applique à lui.
`DATABASE_URL`	Obligatoire (hors Docker)	`postgres://selio_app:MOT_DE_PASSE@hôte:5432/selio`	Fournie automatiquement par docker-compose. À renseigner si vous utilisez une base PostgreSQL 16 managée.
`REDIS_URL`	Recommandé	`redis://hôte:6379`	File de tâches (BullMQ). Sans Redis, les tâches s'exécutent dans l'API (convient pour démarrer).
`SECRETS_ENCRYPTION_KEY`	Obligatoire	`32 octets en base64`	Chiffre les secrets de connexion en base. Générer : node -e "console.log(require('crypto').randomBytes(32).toString('base64'))". À sauvegarder à part : sans elle, les secrets chiffrés sont perdus.

3. Serveur API (réseau et sécurité)

Même fichier .env que ci-dessus (API et worker).

VARIABLE	STATUT	EXEMPLE / FORMAT	À QUOI ELLE SERT
`PUBLIC_WEB_URL`	Obligatoire	`https://votre-site.netlify.app`	Adresse du site web (retours de paiement, liens).
`CORS_ORIGINS`	Obligatoire	`https://votre-site.netlify.app`	Origines autorisées à appeler l'API avec cookies (plusieurs : séparées par des virgules).
`COOKIE_SECURE`	Obligatoire	`true`	Obligatoire en production (HTTPS). Le serveur refuse de démarrer sinon.
`TRUST_PROXY`	Recommandé	`true`	Derrière Caddy/Nginx, pour lire la vraie adresse IP (limites de débit, audit).
`NODE_ENV`	Obligatoire	`production`	Active les garde-fous de production (pas d'IA simulée, cookies sécurisés).
`OPERATOR_EMAILS`	Recommandé	`vous@domaine.fr`	Comptes ayant accès à l'administration Selio.
`PORT / HOST / LOG_LEVEL`	Facultatif	`8787 / 0.0.0.0 / info`	Valeurs par défaut adaptées.

4. IA : votre modèle personnel (générateur d'annonces, réponses)

Le navigateur n'appelle jamais l'IA : seuls l'API et le worker parlent au serveur Ollama, idéalement par réseau privé (VPN). Sans ces variables, l'IA reste simulée (démo uniquement).

VARIABLE	STATUT	EXEMPLE / FORMAT	À QUOI ELLE SERT
`AI_PROVIDER`	Obligatoire	`ollama`	« mock » (simulateur) est interdit en production.
`OLLAMA_BASE_URL`	Obligatoire	`http://10.0.0.5:11434`	Adresse (privée) du PC qui fait tourner Ollama.
`OLLAMA_MODEL`	Obligatoire	`qwen2.5:7b-instruct`	Modèle de texte : réponses aux acheteurs, descriptions. Installer : ollama pull qwen2.5:7b-instruct
`OLLAMA_VISION_MODEL`	Recommandé	`qwen2.5vl:7b (ou llama3.2-vision)`	NOUVEAU — modèle de vision : le générateur lit réellement les photos. Sans lui, l'annonce est rédigée à partir des indications seulement (signalé à l'écran). Installer : ollama pull qwen2.5vl:7b
`OLLAMA_AUTH_HEADER`	Facultatif	`Bearer xxxxx`	NOUVEAU — en-tête d'autorisation si une passerelle protège Ollama.
`AI_HEALTH_TOKEN`	Recommandé	`jeton long (≥ 16 caractères)`	Protège l'URL de supervision /api/ai/health.
`ALLOW_AI_FALLBACK`	Obligatoire	`false`	En production, jamais de texte simulé présenté comme venant du modèle.

VARIABLE	STATUT	EXEMPLE / FORMAT	À QUOI ELLE SERT
`AI_TIMEOUT_MS`, `AI_MAX_CONCURRENCY`, `AI_CONTEXT_LIMIT`, `AI_MAX_OUTPUT_TOKENS`, `AI_QUEUE_MAX_PENDING`, `AI_CIRCUIT_FAILURES`, `AI_CIRCUIT_COOLDOWN_MS`	Facultatif	`45000, 2, 8192, 700, 100, 3, 60000`	Réglages de performance : valeurs par défaut prudentes, à ajuster après « pnpm ai:diagnose ».

5. Paiement des abonnements (Stripe)

Uniquement si vous facturez des abonnements. Les clés se trouvent dans le tableau de bord Stripe › Développeurs. Commencez en mode test.

VARIABLE	STATUT	EXEMPLE / FORMAT	À QUOI ELLE SERT
`STRIPE_SECRET_KEY`	Facultatif	`sk_test_…`	Clé secrète Stripe (mode test d'abord).
`STRIPE_WEBHOOK_SECRET`	Facultatif	`whsec_…`	Signature des événements envoyés par Stripe à /api/billing/webhook.
`STRIPE_PRICE_STARTER` / `STRIPE_PRICE_PRO`	Facultatif	`price_…`	Identifiants des tarifs créés dans Stripe.
`STRIPE_LIVE_ALLOWED`	Facultatif	`false`	Passer à true seulement après validation en mode test.

6. Worker (tâches planifiées)

Mêmes variables base / Redis / chiffrement / IA que l'API. Les cadences ont des valeurs par défaut.

VARIABLE	STATUT	EXEMPLE / FORMAT	À QUOI ELLE SERT
`CRON_AUTOMATION_TICK`, `CRON_JOBS_PROCESS`, `CRON_CONNECTOR_SYNC`, `CRON_RADAR`, `CRON_RETENTION`	Facultatif	`*/5 * * * *`, `* * * * *`, `*/15 * * * *`, `0 * * * *`, `0 3 * * *`	Expressions cron (UTC) : automatisations, file, synchronisation, radar, rétention.

Réglages faits dans l'application (pas des clés)

- **Extension navigateur** — Pas de variable : dans Selio › Paramètres › Connexions, créez un jeton d'extension, puis collez l'adresse de l'API et le jeton dans les options de l'extension.
- **Achat auto** — Page « Achat auto » : seuil de note (85 conseillé), plafond par jour, prix maximal par article, délai d'annulation. Désactivé par défaut.
- **Coût réel d'achat** — Page « Achat auto » : protection acheteurs = part fixe + pourcentage (par défaut 0,70 € + 5 %, modifiable).
- **Dressing intelligent** — Republication conseillée après 7 jours (réglable : 5 à 30 jours).
- **Assistant IA** — Paramètres › Assistant IA : ton, signature, validation avant envoi, quota mensuel.

Inutile

- Aucune clé Vinted : Vinted ne propose pas d'API publique pour les vendeurs. La lecture et l'aide à la publication passent par l'extension Selio, dans votre navigateur, avec votre session.
- Aucune clé OpenAI ou autre IA payante : Selio est prévu pour votre modèle personnel (Ollama).
- Aucun secret dans Netlify : seules les deux variables VITE_ ci-dessus, qui sont publiques.
- Aucune clé Supabase (anon / service) : la base est un PostgreSQL 16 standard (Docker sur le serveur). Vous pouvez l'héberger chez Supabase ou un autre Postgres managé : seule DATABASE_URL change.

Mise en ligne pas à pas

Démo seule (aucune clé)

1. Télécharger livraison/selio-demo-netlify.zip depuis la branche claude/gracious-wright-rf41nt du dépôt selio sur GitHub.
2. Le décompresser, puis glisser le DOSSIER obtenu sur https://app.netlify.com/drop (ou, pour mettre à jour un site existant : Netlify › Deploys › glisser le dossier).
3. C'est en ligne : la démo tourne dans le navigateur, données fictives.

Site relié à GitHub (mises à jour automatiques)

1. Netlify › Add new site › Import from Git › dépôt selio › branche claude/gracious-wright-rf41nt.
2. Netlify lit netlify.toml : commande « pnpm build:web », dossier publié apps/web/dist. Rien à changer.
3. Ajouter VITE_API_BASE_URL quand l'API sera en ligne (sinon : démo seule).

Mode connecté (vrais comptes, base de données)

1. Louer un serveur Linux (VPS) et y installer Docker.
2. Cloner le dépôt, copier apps/api/.env.example vers deploy/.env et remplir les sections 2, 3 et 4.
3. docker compose -f deploy/docker-compose.prod.yml up -d --build (migrations comprises, dont la 0003 de cette version).
4. Mettre Caddy devant l'API (deploy/Caddyfile) pour le HTTPS, puis renseigner VITE_API_BASE_URL dans Netlify.

Brancher votre modèle IA

1. Sur le PC IA : installer Ollama, puis ollama pull qwen2.5:7b-instruct et ollama pull qwen2.5vl:7b.
2. Renseigner AI_PROVIDER=ollama, OLLAMA_BASE_URL, OLLAMA_MODEL, OLLAMA_VISION_MODEL côté API et worker.
3. Lancer pnpm ai:diagnose, puis vérifier Paramètres › Assistant IA (« Opérationnel »).

Sécurité. Ne commitez jamais de fichier `.env`. Sauvegardez `SECRETS_ENCRYPTION_KEY` séparément de la base. N'exposez pas le port 11434 d'Ollama sur Internet. En production, le serveur refuse de démarrer si `COOKIE_SECURE` n'est pas `true`, si `AI_PROVIDER=mock` ou si `ALLOW_AI_FALLBACK=true`.

Source : docs/CLES-ET-CONFIGURATION.md · modèles de fichiers : apps/api/.env.example, apps/worker/.env.example, apps/web/.env.example.